Web Application Development

Lecture 5: State & Data Flow

SWE 381 – Web Application Development

Dr. Ohoud Alharbi Dr. Achraf Gazdar

Department of Software Engineering
College of Computer and Information Sciences
King Saud University

Term 2 – 2025/2026

Learning Objectives

By the end of this lecture, you will be able to:

- Explain what **lifting state up** means and when to use it
- Implement the **3-step lifting pattern** in a React component tree
- Enable **sibling communication** through a shared parent
- Pass **callback props** to let children update parent state
- State and apply the **immutability rule** for all state updates
- **Add, remove,** and **update** items in an array state immutably
- **Update objects** in state using the spread operator correctly

Agenda

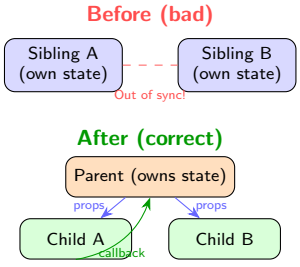
1. Lifting State Up
2. Sibling Communication via Parent
3. Callback Props Pattern
4. Immutability Rule
5. Array Operations – Add / Remove / Update
6. Updating Objects with Spread
7. Practical Examples & Summary
8. Recap

Outline

What is Lifting State Up?

React – Sharing State Between Components

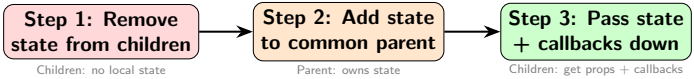
In React, sharing state is accomplished by **moving it up to the closest common ancestor** of the components that need it. This is called **"lifting state up"**. There should be a **single "source of truth"** for any data that changes in a React application.



When do you need to lift state?

- Two or more components need to **reflect the same data**
- One component's action must **affect another** component
- You find yourself **duplicating state** across siblings
- Components need to stay **synchronised**

The 3-Step Lifting Pattern



Before lifting – each panel has own state

Two panels can be expanded simultaneously because neither knows about the other's state.

After lifting – parent controls both

The parent holds one `activeIndex`. Only one panel can be open at a time, guaranteed by a single source of truth.

Single source of truth

For each piece of state there is **one specific component** that owns it. Other components receive it via props – never duplicate it.

Lifting State – Before: State Duplicated in Each Child

```jsx
import { useState } from 'react';

// BEFORE lifting: each Panel manages its own open/closed state
function Panel({ title, children }) {
  const [isOpen, setIsOpen] = useState(false); // <-- local state

  return (
    <div style={{ border: '1px solid #ccc', margin: '8px', padding: '8px' }}>
      <h3>{title}</h3>
      {isOpen
        ? <p>{children}</p>
        : <button onClick={() => setIsOpen(true)}>Show</button>
      }
    </div>
  );
}

// Parent has NO control -- both can be open at the same time!
function Accordion() {
  return (
    <>
      <h2>FAQ</h2>
      <Panel title="What is React?">
        React is a JavaScript library for building UIs.
      </Panel>
      <Panel title="What is useState?">
        useState is a React Hook to track state.
      </Panel>
    </>
  );
}
```

Lifting State – After: Parent Owns the State

```jsx
import { useState } from 'react';

// AFTER: Panel receives state as props
// -- no local useState any more
function Panel({
  title, children, isOpen, onShow }) {
  return (
    <div style={{
      border: '1px solid #ccc',
      margin: '8px', padding: '8px' }}>
      <h3>{title}</h3>
      {isOpen
        ? <p>{children}</p>
        : <button onClick={onShow}>
            Show
          </button>
      }
    </div>
  );
}
```

```jsx
// Parent owns the state
function Accordion() {
  const [activeIndex, setActiveIndex]
    = useState(0);
  return (
    <>
      <h2>FAQ</h2>
      <Panel
        title="What is React?"
        isOpen={activeIndex === 0}
        onShow={() => setActiveIndex(0)}>
        A JS library for building UIs.
      </Panel>
      <Panel
        title="What is useState?"
        isOpen={activeIndex === 1}
        onShow={() => setActiveIndex(1)}>
        useState tracks React state.
      </Panel>
    </>
  );
}
```

Lifting State – Visual Result

activeIndex = 0 (first panel open)

FAQ

What is React?
React is a JavaScript library...

What is useState?
Show

activeIndex = 1 (second panel open)

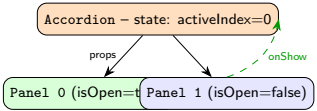
FAQ

What is React?
Show

What is useState?
useState is a React Hook...

What changed:

- `Panel` lost its `useState`
- `Panel` received `isOpen` as a prop
- `Panel` received `onShow` callback
- `Accordion` gained `activeIndex` state
- Only one panel can be open at a time

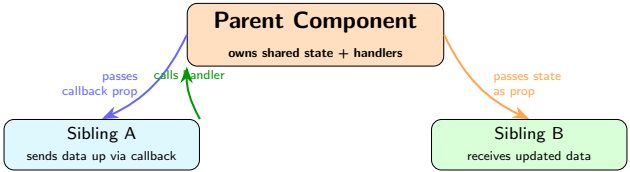


Outline

Sibling Communication – The Core Pattern

React – Sharing State

When you want to coordinate two components, move their state to their **common parent**. Then pass the information down through props from the common parent. The parent becomes the **"middleman"** that coordinates the siblings.



Sibling Communication – Search Filter Example

```jsx
import { useState } from 'react';

// Sibling A: sends user input UP
function SearchBar({ query, onSearch }) {
  return (
    <input
      placeholder="Search courses..."
      value={query}
      onChange={
        e => onSearch(e.target.value)
      }
    />
  );
}

// Sibling B: receives data DOWN
function CourseList({ courses }) {
  return (
    <ul>
      {courses.map(c =>
        <li key={c.id}>{c.title}</li>
      )}
    </ul>
  );
}
```

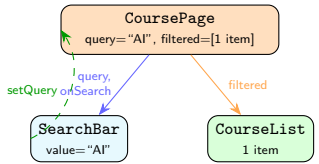
```jsx
// Parent: owns state, coordinates both
function CoursePage() {
  const [query, setQuery] = useState("");
  const allCourses = [
    { id: 1, title: "Web App Dev" },
    { id: 2, title: "SW Design" },
    { id: 3, title: "AI and ML" },
  ];

  const filtered = allCourses.filter(c =>
    c.title.toLowerCase()
      .includes(query.toLowerCase())
  );

  return (
    <div>
      <SearchBar
        query={query}
        onSearch={setQuery} />
      <CourseList courses={filtered} />
    </div>
  );
}
```

Sibling Communication – Step-by-Step Trace

User types "AI"

AI

- AI and Machine Learning



Execution trace:

1. User types `"AI"` in `SearchBar` input
2. `onChange` calls `onSearch("AI")`
3. Parent receives call → `setQuery("AI")`
4. `CoursePage` re-renders with `query = "AI"`
5. `filtered` computed: 1 course matches
6. `SearchBar` re-renders with `query="AI"`
7. `CourseList` re-renders with 1-item array

Key insight

The siblings **never communicate directly**. All data goes through the parent.

Outline

Callback Props – Recap and Deeper Look

Callback Props Pattern

A **callback prop** is a function passed from a parent to a child via props. The child calls this function when something happens, effectively sending data or events **back up** to the parent. This is how React achieves **bidirectional communication** in a unidirectional data-flow system.

Scenario	Parent passes	Child calls when
Search input	`onSearch`	User types
Delete item	`onDelete`	User clicks X
Toggle item	`onToggle`	User clicks checkbox
Form submit	`onSubmit`	Form submitted
Select item	`onSelect`	User picks row
Rating change	`onRate`	Star clicked

Naming convention

Always prefix callback props with `on`: `onClick`, `onChange`, `onDelete`, `onClose`, `onSelect`.

Handler naming

Inside the parent, name the handler `handle` + **Event**: `handleDelete`, `handleClose`, `handleSelect`.

Callback Props – Grade Manager Example

```jsx
import { useState } from 'react';

// Child: renders one student row
function StudentRow({
  student, onDelete, onTogglePass }) {
  return (
    <tr style={{
      color: student.passed
        ? 'green' : 'red' }}>
      <td>{student.name}</td>
      <td>{student.grade}</td>
      <td>
        {student.passed ? 'Pass' : 'Fail'}
      </td>
      <td>
        <button onClick={
          () => onTogglePass(student.id)}>
          Toggle
        </button>
        <button onClick={
          () => onDelete(student.id)}>
          Delete
        </button>
      </td>
    </tr>
  );
}
```

```jsx
// Parent: owns all state and handlers
function GradeManager() {
  const [students, setStudents] = useState([
    { id:1, name:'Ahmed',
      grade:88, passed:true  },
    { id:2, name:'Sara',
      grade:55, passed:false },
    { id:3, name:'Khalid',
      grade:72, passed:true  },
  ]);

  const handleDelete = (id) =>
    setStudents(
      students.filter(s => s.id !== id));

  const handleTogglePass = (id) =>
    setStudents(students.map(s =>
      s.id === id
        ? { ...s, passed: !s.passed }
        : s
    ));

  return (
    <table><tbody>
      {students.map(s => (
        <StudentRow key={s.id} student={s}
          onDelete={handleDelete}
          onTogglePass={handleTogglePass}/>
      ))}
```

Callback Props – Visual Output

Grade Manager

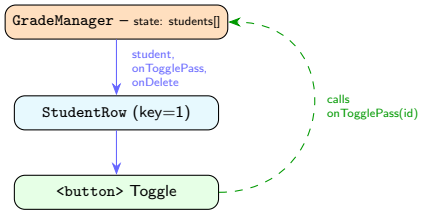
Name	Grade	Status	Actions
Ahmed	88	Pass	Toggle Delete
Sara	55	Fail	Toggle Delete
Khalid	72	Pass	Toggle Delete

After clicking Toggle on Sara:

Updated state

Name	Grade	Status	Actions
Ahmed	88	Pass	Toggle Delete
Sara	55	Pass	Toggle Delete
Khalid	72	Pass	Toggle Delete

Data flow recap:



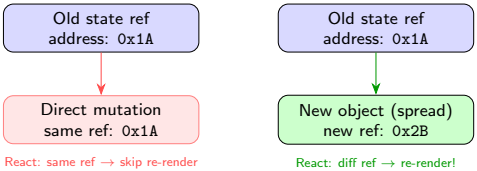
Outline

The Immutability Rule – Why It Matters

React – Updating Objects in State

Although objects in React state are technically mutable, you should **treat them as if they were immutable** – like numbers, booleans, and strings. Instead of mutating them, you should always **replace** them with a new copy.

How React detects changes:



Benefits of immutability:

- React detects changes via **reference comparison**
- Guarantees **predictable re-renders**
- Enables **time-travel debugging**
- Makes **undo/redo** trivial
- Prevents **hidden side effects**
- Supports **pure component optimisations**

Reference equality

JavaScript compares objects/arrays by **reference**, not by value. `{a:1} === {a:1}` is `false` – two different objects!

Mutation vs Immutable Update – Side by Side

WRONG – direct mutation:

```
// Object mutation -- NO re-render!
const [student, setStudent] =
  useState({ name: 'Ahmed', gpa: 3.5 });

// WRONG: modifying the existing object
student.gpa = 4.0;
setStudent(student);
// React sees the same reference
// UI stays stale

// Array mutation -- NO re-render!
const [list, setList] = useState([1,2,3]);
list.push(4);          // WRONG: mutates!
setList(list);         // same ref -> no update

list.splice(0, 1);     // WRONG: mutates!
list[0] = 99;          // WRONG: mutates!
```

CORRECT – create new value:

```
// Object -- spread creates new ref
const [student, setStudent] =
  useState({ name: 'Ahmed', gpa: 3.5 });

// CORRECT: new object via spread
setStudent({ ...student, gpa: 4.0 });
// New reference -> React re-renders

// Array -- spread / filter / map
const [list, setList] = useState([1,2,3]);

// ADD: spread + new item
setList([...list, 4]);

// REMOVE: filter
setList(list.filter(x => x !== 1));

// UPDATE: map
setList(list.map(x => x === 1 ? 99 : x));
```

Operation	Mutating (AVOID)	Immutable (USE)
Add to array	`push(), unshift()`	`[...arr, item]`
Remove from arr	`pop(), splice()`	`arr.filter(...)`
Update in arr	`arr[i] = x`	`arr.map(...)`
Update object	`obj.key = val`	`{...obj, key:val}`

Outline

Adding Items to Array State

React – Updating Arrays in State

Instead of using `push()` (which mutates), create a new array which contains the existing items and a new item at the end. The easiest way is to use the **array spread syntax**.

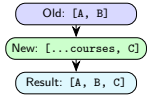
```
import { useState } from 'react';

function CourseList() {
  const [courses, setCourses] = useState([
    { id: 1, title: "Web App Dev" },
    { id: 2, title: "SW Design" },
  ]);
  const [input, setInput] = useState("");

  // ADD at the END -- spread + new item
  const addCourse = () => {
    if (!input.trim()) return;
    setCourses([
      ...courses,
      { id: Date.now(), title: input }
    ]);
    setInput("");
  };

  // ADD at the BEGINNING -- new item + spread
  const addFirst = () => {
    setCourses([
      { id: Date.now(), title: input },
      ...courses
    ]);
```

After adding "AI Basics"

AI Basics

Add Last | Add First

- Web App Dev
- SW Design
- AI Basics



Removing Items from Array State

React – Updating Arrays in State

The easiest way to remove an item from an array is to **filter it out**. You produce a new array that will not contain that item using the `filter` method.

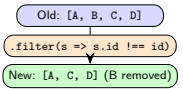
```jsx
import { useState } from 'react';

function StudentList() {
  const [students, setStudents] = useState([
    { id: 1, name: 'Ahmed',  gpa: 3.8 },
    { id: 2, name: 'Sara',   gpa: 3.5 },
    { id: 3, name: 'Khalid', gpa: 3.2 },
    { id: 4, name: 'Noura',  gpa: 2.9 },
  ]);

  // REMOVE by id -- filter creates new array
  const removeStudent = (id) => {
    setStudents(students.filter(s => s.id !== id));
  };

  // REMOVE all below threshold
  const removeFailing = () => {
    setStudents(students.filter(s => s.gpa >= 3.0));
  };

  return (
    <div>
      <button onClick={removeFailing}>
        Remove Failing (GPA less than 3.0)
      </button>
```

After Remove Failing click

Remove Failing

- Ahmed (3.8) X
- Sara (3.5) X
- Khalid (3.2) X

Noura removed



Updating Items in Array State

React – Updating Arrays in State

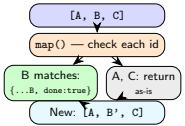
To update one or more items, use `map()` to produce a new array. For the item(s) you want to change, you can make changes (or return a new object). For the items you don't want to change, return them as-is.

```
import { useState } from 'react';

function TodoList() {
  const [todos, setTodos] = useState([
    { id: 1, text: "Learn React", done: false },
    { id: 2, text: "Build app",   done: false },
    { id: 3, text: "Deploy",      done: false },
  ]);

  // TOGGLE done status for one item
  const toggleTodo = (id) => {
    setTodos(todos.map(todo =>
      todo.id === id
        ? { ...todo, done: !todo.done } // new obj
        : todo                          // unchanged
    ));
  };

  return (
    <ul>
      {todos.map(todo => (
        <li
          key={todo.id}
          style={{ textDecoration:
            todo.done ? 'line-through' : 'none' }}
          onClick={() => toggleTodo(todo.id)}>
```

After clicking item 2

- Learn React
- Build app (done)
- Deploy

The map + spread pattern:



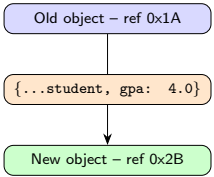
Outline

Updating Object State – The Spread Pattern

React – Updating Objects in State

The **spread syntax** (`{...obj}`) creates a **shallow copy** of the object, then you add or override only the properties you want to change. This produces a new object with a new reference.

```jsx
import { useState } from 'react';

function StudentProfile() {
  const [student, setStudent] = useState({
    name:  "Ahmed",
    email: "ahmed@ksu.edu.sa",
    gpa:   3.5,
    dept:  "SWE",
    year:  3
  });

  // Update only ONE field -- spread rest
  const handleNameChange = (e) => {
    setStudent({
      ...student,           // keep all other fields
      name: e.target.value // override only name
    });
  };

  const handleGpaChange = (e) => {
    setStudent({
      ...student,
      gpa: parseFloat(e.target.value)
    });
  };
```



Order matters

Place the override **after** the spread: `{...obj, key: newVal}`. If before, the spread would overwrite it back.

Updating Multiple Fields at Once

One handler for all fields (DRY):

```jsx
import { useState } from 'react';

function EditForm() {
  const [form, setForm] = useState({
    name:    "",
    email:   "",
    phone:   "",
    dept:    "SWE"
  });

  // One handler using input name attribute
  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm({
      ...form,
      [name]: value   // computed property name
    });
  };

  return (
    <form>
      <input name="name"  value={form.name}
             onChange={handleChange}
             placeholder="Name" />
      <input name="email" value={form.email}
             onChange={handleChange}
             placeholder="Email" />
      <input name="phone" value={form.phone}
             onChange={handleChange}
             placeholder="Phone" />
      <p>
```

How computed property name works:

```jsx
// e.target.name = "email"
// e.target.value = "a@ksu.sa"

const { name, value } = e.target;
// name  = "email"
// value = "a@ksu.sa"

setForm({
  ...form,
  [name]: value
  // same as: email: "a@ksu.sa"
});
```

Computed property names

`[name]: value` uses the **value of** `name` as the key. This allows one handler to update any field of the object.

Real-world pattern

Using one `handleChange` function for all inputs is a very common React pattern. The `name` attribute on each input maps to the state field.

Updating Objects Inside an Array

Combine map + spread

To update a single object inside a state array: use `map()` to walk the array, and the **spread operator** to clone and patch only the matching item. Other items are returned unchanged.

```
const [students, setStudents] = useState([
  { id: 1, name: "Ahmed",
    gpa: 3.5, dept: "SWE" },
  { id: 2, name: "Sara",
    gpa: 3.8, dept: "CS"  },
  { id: 3, name: "Khalid",
    gpa: 3.2, dept: "SWE" },
]);

// Update ONE field of ONE student
const updateGpa = (id, newGpa) => {
  setStudents(students.map(s =>
    s.id === id
      ? { ...s, gpa: newGpa }
      : s
  ));
};
```

```
// Update MULTIPLE fields at once
const updateStudent = (id, changes) => {
  setStudents(students.map(s =>
    s.id === id
      ? { ...s, ...changes }
      : s
  ));
};

// Usage examples
updateGpa(2, 4.0);
// Sara: gpa -> 4.0, others unchanged

updateStudent(1, { gpa: 3.9, dept: "CS" });
// Ahmed: gpa AND dept change together
```

Outline

Complete Example – Course Registration App

```jsx
import { useState } from 'react';

// Child: one course row
function CourseRow({ course, onEnrol, onDrop }) {
  return (
    <tr>
      <td>{course.title}</td>
      <td>{course.credits}</td>
      <td>
        {course.enrolled
          ? 'Enrolled' : 'Available'}
      </td>
      <td>
        {course.enrolled
          ? <button onClick={
              () => onDrop(course.id)}>
              Drop
            </button>
          : <button onClick={
              () => onEnrol(course.id)}>
              Enrol
            </button>
        }
      </td>
    </tr>
  );
}
```

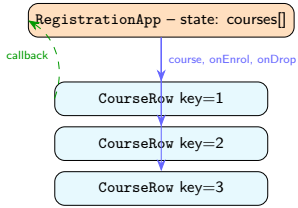
```jsx
// Parent: owns all state
function RegistrationApp() {
  const [courses, setCourses] = useState([
    { id:1, title:"Web App Dev",
      credits:3, enrolled:false },
    { id:2, title:"SW Design",
      credits:3, enrolled:false },
    { id:3, title:"AI Basics",
      credits:2, enrolled:false },
  ]);

  const handleEnrol = (id) =>
    setCourses(courses.map(c =>
      c.id === id
        ? { ...c, enrolled: true } : c));

  const handleDrop = (id) =>
    setCourses(courses.map(c =>
      c.id === id
        ? { ...c, enrolled: false } : c));

  const enrolled = courses.filter(
    c => c.enrolled);
  const totalCred = enrolled.reduce(
    (s, c) => s + c.credits, 0);

  return (
    <div>
      <h2>{totalCred} credits enrolled</h2>
      <table><tbody>
        {courses.map(c => (
          <CourseRow key={c.id} course={c}
            onEnrol={handleEnrol}
            onDrop={handleDrop} />
        ))}
```

Registration App – Output and Architecture

After enrolling Web App Dev and AI Basics

Course Registration – 5 credits enrolled

Title	Cr	Status	Action
Web App Dev	3	Enrolled	Drop
SW Design	3	Available	Enrol
AI Basics	2	Enrolled	Drop



Patterns used

Lifting state, callback props, immutable map+spread update, derived values (no extra state).

Immutability Cheatsheet – Quick Reference

Object updates:

```
// Update one field
setObj({ ...obj, field: newVal });

// Update multiple fields
setObj({ ...obj, a: 1, b: 2 });

// Computed field name
setObj({ ...obj, [key]: value });

// Initialise from prop (starting value only)
const [x, setX] = useState(prop.x);
```

Nested object:

```
// Spread each level that contains a change
setUser({
  ...user,
  address: {
    ...user.address,
    city: "Riyadh"
  }
});
```

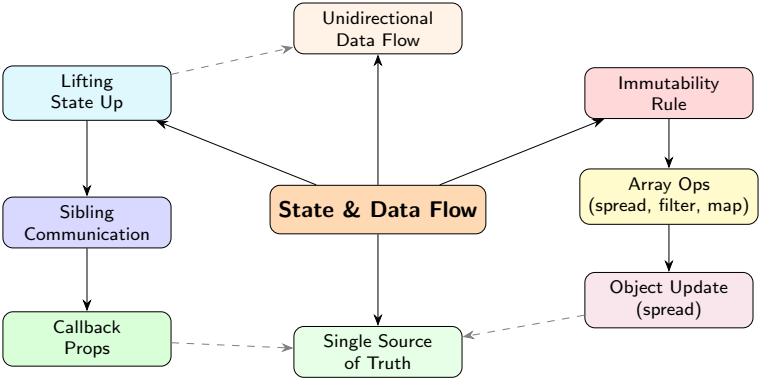
Array updates:

```
// Add at end
setArr([...arr, newItem]);

// Add at beginning
setArr([newItem, ...arr]);

// Remove by id
setArr(arr.filter(x => x.id !== id));

// Update one item
setArr(arr.map(x =>
  x.id === id ? { ...x, field: val } : x
));

// Replace entire array
setArr([a, b, c]);

// Clear array
setArr([]);
```

Never use these on state

push, pop, shift, unshift, splice, sort, reverse – all **mutate** the original array. Avoid them on state arrays.

State & Data Flow – Concept Map



Common Mistakes and How to Fix Them

Mistake 1 – Duplicating state:

```
// WRONG: each component tracks same data
function A() { const [x, setX] = useState(0); }
function B() { const [x, setX] = useState(0); }
// A and B are out of sync!

// FIX: lift to common parent
function Parent() {
  const [x, setX] = useState(0);
  return <><A x={x}/><B x={x} setX={setX}/></>;
}
```

Mistake 2 – Mutating state:

```
// WRONG
const [items, setItems] = useState([...]);
items.push(newItem); setItems(items); // stale!

// FIX
setItems([...items, newItem]); // new array
```

Mistake 3 – Derived value in state:

```
// WRONG: total out of sync
const [items, setItems] = useState([]);
const [total, setTotal] = useState(0);
// Must remember to update both!

// FIX: derive total from items
const total = items.reduce(
  (s, i) => s + i.price, 0
);
```

Mistake 4 – Object mutation:

```
// WRONG
const [user, setUser] = useState({name:'A'});
user.name = 'B';  // mutates!
setUser(user);    // same ref -> no re-render

// FIX
setUser({ ...user, name: 'B' }); // new obj
```

Outline

All Patterns at a Glance

```jsx
// ---- LIFTING STATE UP ----
function Parent() {
  const [shared, setShared] = useState(0);
  return (
    <>
      <ChildA value={shared} />
      <ChildB onUpdate={setShared} />
    </>
  );
}

// ---- CALLBACK PROPS ----
function ChildB({ onUpdate }) {
  return (
    <button onClick={() => onUpdate(42)}>
      Update
    </button>
    // callback fires
    // -> parent state changes
    // -> ChildA re-renders
  );
}
```

```jsx
// ---- IMMUTABLE ARRAY ----
setArr([...arr, newItem]);            // add
setArr(arr.filter(
  x => x.id !== id));                 // remove
setArr(arr.map(x =>
  x.id===id
    ? {...x, ...changes} : x));       // update

// ---- IMMUTABLE OBJECT ----
setObj({ ...obj, field: newValue }); // one field
setObj({ ...obj, [key]: value });    // computed

// ---- SIBLING COMMUNICATION ----
// Sibling A calls callback
// -> Parent updates state
// -> Parent passes new value
//    to Sibling B as prop
```

Lecture 5 – Key Takeaways

1. **Lifting state up** means moving shared state to the closest common ancestor – creating a **single source of truth**
2. The **3-step pattern**: remove state from children → add to parent → pass down as props + callbacks
3. **Siblings communicate through their parent**: A calls a callback, parent updates state, B receives updated props
4. **Callback props** use the naming convention `on` + `Event` (e.g. `onDelete`, `onSelect`)
5. React uses **reference equality** to detect changes – direct mutation is invisible to React
6. **Immutability rule**: always produce a **new** value using spread / filter / map; never push, splice, or assign directly to state
7. **Add** to array: `[...arr, item]` **Remove**: `arr.filter()` **Update**: `arr.map()`
8. **Update object field**: `{...obj, field: newVal}` – spread keeps all other fields intact
9. **Update object in array**: chain `map()` with spread inside the callback

Practice Exercises

1. Build an **Accordion** component with two `Panel` children. Store `activeIndex` in the parent so only one panel can be open at a time (React's lifting state up example).
2. Create a **Search + List** pair: a `SearchBar` component and a `ItemList` component. Lift the `query` state to a parent `App` so the list filters as the user types.
3. Add a **Delete** button to each list item using the callback props pattern. The parent's handler removes the item immutably using `filter()`.
4. Build a **Student Profile Editor** that holds a student object in state. Use the spread operator to update only the gpa field when a button is clicked. Verify the name and dept remain unchanged.
5. Create a **To-Do List** where items can be added (spread), removed (filter), and marked done (map + spread). Never use `push` or `splice`.
6. **Sibling communication**: build two components – a `ScoreInput` (slider) and a `GradeDisplay` (shows A/B/C/D/F). Lift the score state to a parent so both stay synchronised.
7. **Challenge**: extend the Course Registration App from the lecture – add a **Drop All** button, a **credit limit** (max 9), and a message when the limit is reached. All operations must be immutable.